DSTM Airport Simulation Example

Nicholas Cassarino

June 12, 2026

Warning, this text is AI-generated.

1 Airport Simulation Description

The simulated world consists of a fixed number of *airports* partitioned evenly across MPI ranks, where each rank owns `na` local airports and the global total is `num_ranks` × `na`.

Each airport is assigned one of three tiers, `HUB`, `REGIONAL`, or `LOCAL`, with hubs being rarer but more heavily trafficked. The tier assignment is deterministic from a shared random seed so that every rank agrees on the global tier map. Tier influences two probabilities:

- **Send probability** (`tier_send_prob`): how likely an airport is to inject a new outbound flight in any given timestep (HUB: 80%, REGIONAL: 40%, LOCAL: 15%).
- **Receive weight** (`tier_recv_weight`): the weighted probability of being chosen as a destination by some other airport (HUB: 8, REGIONAL: 3, LOCAL: 1).

At each timestep, every airport may:

1. Inject a new flight bound for a randomly chosen destination on another rank (weighted by tier).
2. Forward in-transit "connecting" flights toward their next hop, using a mesh topology with a configurable layover threshold to decide whether a flight goes direct or requires an intermediate rank.
3. Land flights that have arrived at their final destination, performing a configurable amount of synthetic CPU work (`--work`) per arrival to simulate per-event processing cost.

Each rank maintains an inbox of `NUM_GATES` gate slots into which remote ranks deposit batches of flights, and an outbox indexed by destination rank used to stage outgoing flight batches. Per-airport counters track departures, arrivals, and connecting transits, and are gathered on rank 0 at the end of the run to produce a summary.

The simulation has three command-line knobs that matter for scaling experiments: `--na N` (airports per rank), `--time T` (number of timesteps), and `--work W` (per-arrival compute work).

2 DSTM vs MPI version

Two implementations of the simulation exist that differ *only* in how inter-rank communication is performed. All simulation logic (flight injection, routing, tier behavior, RNG seeding, and per-airport counters) is the same between the two.

DSTM version. The original implementation uses the Distributed Shared Transactional Memory (DSTM) runtime. Each rank's inbox of `NUM_GATES` gate slots lives in a globally addressable distributed array (`dstm_darray_t`). Communication is one-sided and asynchronous:

- To send a batch, a rank calls `dstm_darray_put` on the destination's gate slot, optionally tagged with a per-link distance (`delta`) that controls when the value becomes visible on the consumer side.
- To receive, a rank calls `dstm_darray_get_or_default` on each of its own gate slots, defaulting to an empty gate if no producer has written one for the current timestep.
- Time advances via DSTM's logical-time model; a barrier at end of run guarantees all in-flight PUTs are settled before counts are gathered.

MPI version. The MPI port replaces the DSTM darray with explicit non-blocking point-to-point messaging. Each timestep performs one exchange round:

- Each rank posts `MPI_Irecv` from every other rank into a local inbox buffer.
- Each rank posts `MPI_Isend` of its outbox (one `gate_slot_t` per destination) to every other rank.
- Two `MPI_Waitall` calls flush the receive and send request lists.
- A final `MPI_Barrier` synchronizes ranks before the next timestep, mirroring DSTM's per-step time advancement.

To preserve DSTM's "data sent at time t is visible at time $t+1$" semantics, the MPI version persists the received inbox between timesteps: processing at step t consumes the inbox that arrived during step $t-1$'s exchange, and the outbox built during step t's processing is shipped in step t's exchange to be processed at step $t+1$.

Differences worth noting. Two implementation differences remain between the DSTM and MPI versions even after the latency-modeling fix described below:

- **Empty outbox slots are still sent over the wire under MPI.** The symmetric all-pairs exchange requires every rank to post a matching send for every posted receive, so empty `gate_slot_t`s travel as zero-flight messages. DSTM, by contrast, only PUTs when there is data, and the consumer defaults missing slots to empty. Semantically these are equivalent, but the MPI version moves more bytes on the wire — $O(P^2)$ messages per timestep regardless of traffic density.
- **Synchronization style.** DSTM uses logical-time consistency to coordinate ranks; the MPI port uses an explicit `MPI_Barrier` at the end of each timestep. The MPI barrier is a stricter form of synchronization and may introduce stragglers-dominated cost at high rank counts.

Modeling DSTM's `delta` in MPI. The original DSTM `put` accepts a per-edge `delta` parameter that controls how many timesteps must pass before a written value becomes visible to consumers — effectively modeling per-link network latency. A naive MPI port collapses every `delta` to 1, which means flights in the MPI version land sooner than in the DSTM version: arrival counts inflate and the per-arrival synthetic work (`--work`) fires more often, making the MPI build look like it is doing more useful work per second than it actually is.

To eliminate this bias the MPI version stamps every outgoing flight with an `arrive_time = t + delta`, where `delta` is the same mesh-distance value DSTM would have used. On the receiver

side, flights are held in a per-rank *pending queue* and only processed once the local clock reaches the flight's `arrive_time`. Flights that are still in flight when the simulation ends are discarded, matching DSTM's behavior without a final drain barrier.

What this means for performance comparisons. With the pending-queue change in place, both implementations perform the same amount of simulation work — the same flights are injected, the same number land, and the same per-arrival compute runs — so elapsed time, timesteps-per-second, and flight totals are directly comparable between the two. There are, however, two effects on the MPI side that are worth flagging honestly:

- **Slightly more receiver-side bookkeeping.** The pending queue adds a per-step pass over in-flight flights to check `arrive_time == t`. In practice this is small (a linear scan over a list bounded by $\sim P \cdot$ `GATE_CAPACITY` $\cdot \max(\text{delta})$ entries), but it is non-zero work that the DSTM version offloads onto its runtime.
- **Memory footprint scales with in-flight population.** DSTM stores in-transit values inside its own buffers; the MPI port stores them in the application-visible `pending` queue. At steady state both must hold the same number of in-flight flights, but the MPI version pays for that storage explicitly in user-space heap.

These overheads slightly favor DSTM in any head-to-head timing. Reported metrics should therefore be read as a faithful apples-to-apples comparison of the two communication models, with the caveat that the MPI version absorbs into the application a small amount of bookkeeping that the DSTM runtime hides.

3 Experiment

We conducted a *weak-scaling* study comparing the two implementations. The per-rank airport count is held constant (`na = AIRPORTS_PER_RANK`) while the rank count P varies, so the global problem size grows linearly with P: `total_airports` = `AIRPORTS_PER_RANK` $\cdot P$. Under ideal weak scaling, elapsed time stays flat as P grows — each rank does the same amount of local work regardless of cluster size.

Why weak scaling instead of strong scaling. We initially ran a strong-scaling study (fixed total airports, shrinking `na`) but the results were dominated by communication overhead at moderate-to-large P and did not meaningfully separate the two implementations. The cause is structural: per-rank compute scales as $O(\texttt{na}) = O(1/P)$, while the symmetric all-pairs exchange forces $O(P)$ messages per rank per step regardless of traffic density. Once P is large enough that comm cost exceeds the shrinking compute, both builds collapse for the same reason, masking any difference between their communication models. Weak scaling keeps per-rank compute constant, so the comm-vs-compute ratio is stable across the sweep and the implementations can be compared on equal footing as the cluster grows.

Configuration.

- Airports per rank (fixed): 32
- Timesteps per run: 8
- Per-arrival synthetic work: 1,000 iterations
- Rank sweep: $P \in \{1, 2, 4, 8, 16, 32, 64, 128, 256\}$

- Total airports (varies): $32 \cdot P$, from 32 at P=1 to 8192 at P=256
- Nodes (Cobalt allocation): 8
- Ranks-per-node: $\max(1, P/8)$, distributed via `--map-by ppr:PPN:node`

Builds compared.

- `dstm`: original DSTM-backed communication, at `../airports/airports`.
- `mpi`: faithful MPI port using `Isend`/`Irecv` with per-timestep barrier, at `../mpi_air/airports`.

Results. Figures 1 and 2 show the weak-scaling behavior of the two implementations. Under ideal weak scaling the elapsed-time curve should remain flat as P grows; any upward slope reflects communication and synchronization overhead that the constant per-rank workload cannot amortize. The two builds track each other closely for $P \leq 8$ (within roughly 2× on elapsed time), but the MPI version breaks down sharply at $P = 16$, jumping from ∼30 ms to nearly 2 s — a 60× regression — and continuing to climb to ∼70 s at $P = 128$. The DSTM build degrades far more gracefully, reaching only ∼2.5 s at $P = 256$ (the largest run we could complete; the MPI build did not finish at this size within our time budget). The cliff at $P = 16$ coincides exactly with the transition from one rank per node to multiple ranks per node, suggesting that the symmetric all-pairs exchange begins paying serious intra-node contention and barrier-cost penalties as soon as ranks start sharing a host.

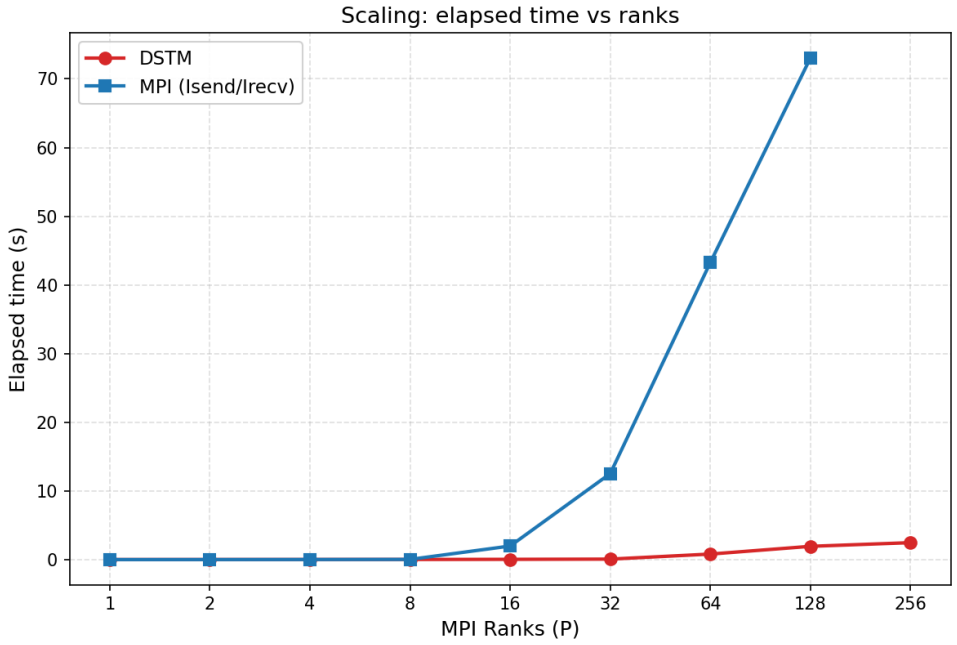


Figure 1: Weak scaling: elapsed simulation time vs. rank count for the DSTM and MPI builds, with the per-rank airport count held fixed at 32. Lower is better; flat is ideal.

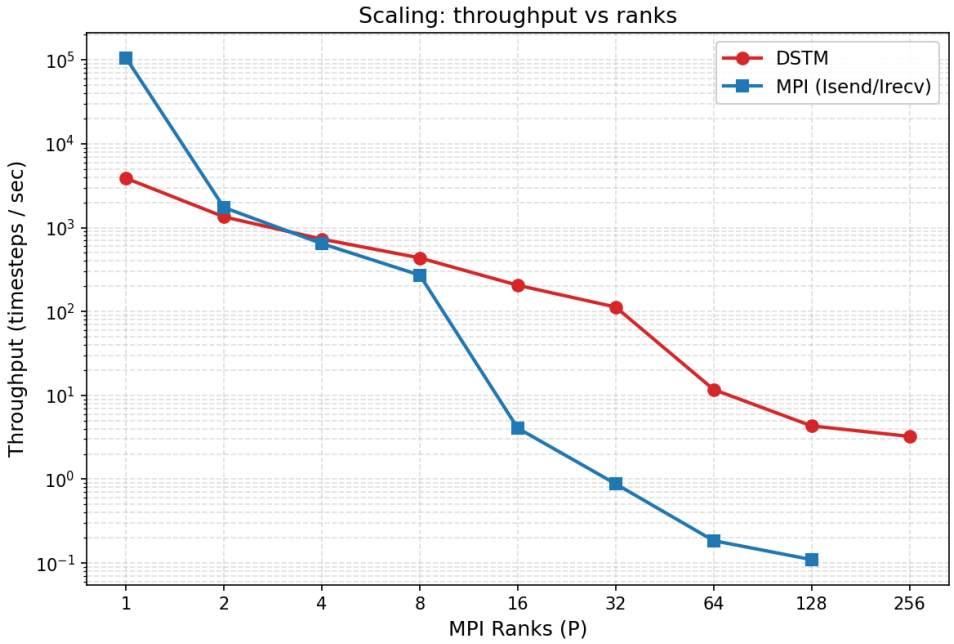


Figure 2: Weak scaling: simulation throughput (timesteps per second) vs. rank count, log–log axes. Higher is better.